

Multiprocess Implementation of String Searching and Compare to Sequential Mode

Cristiano Narducci

E-mail address

cristiano.narducci@edu.unifi.it

Abstract

The work presented introduces a Multiprocessing variant of string searching, utilizing the Levenshtein distance algorithm and the Bitap algorithm for comparison purposes with the correlative sequential version. We compared the two different approach with a fixed length phrase dataset and also with a dedicated string and data set generator function. The final result shows that the two different algorithms can speed up the times of execution using multi processes instead of just one main process. With small dataset length, however all the advantages of the multiprocess version reduce because of the overhead and the context switching of the processor.

Future Distribution Permission

The author(s) of this report give permission for this document to be distributed to Unifi-affiliated students taking future courses.

1. Introduction

1.1. String Searching

The string searching inside a document, or in general in a specific dataset, is a function always needed for the advantage of the users. Many researchers studied the problem, and many others defined dedicated algorithms to optimize and sped up this type of search. The following paper provides two different algorithms and approaches for string searching.

1.2. Levenshtein Distance

In computer science, the Levenshtein distance is a string metric to measure the difference between two sequences of strings. It works by mea-

suring the minimum number of character changes needed to transform the first sentences to the second.[2] The Levenshtein distance between two words a and b , with lengths, respectively, of $|a|$ and $|b|$ is given by the following formulae:

$$\delta = \begin{cases} 0 & \text{if } s_i = t_j \\ 1 & \text{if } s_i \neq t_j \end{cases}$$
$$D(i, j) = \begin{cases} i & \text{if } j = 0 \\ j & \text{if } i = 0 \\ \min \begin{cases} D(i-1, j) + 1 \\ D(i, j-1) + 1 \\ D(i-1, j-1) + \delta \end{cases} & \text{alternately} \end{cases}$$

where i and j are the first "i" and "j" characters of the two strings. The possible edits that are possible to perform are substitution, deletion and insertion.

1.3. Bitap

The bitap algorithm (also known as the shift-or, shift-and or Baeza-Yates–Gonnet algorithm) is an **approximate string matching algorithm**. The algorithm tells whether a given text contains a substring which is "approximately equal" to a given pattern, where approximate equality is defined in terms of **Levenshtein distance** – if the substring and pattern are within a given distance k of each other, then the algorithm considers them equal. The algorithm begins by precomputing a set of bitmasks containing one bit for each element of the pattern. Then it is able to do most of the work with bitwise operations, which are extremely fast. The bitap algorithm is perhaps best

known as one of the underlying algorithms of the Unix utility **agrep**. The bitap algorithm for exact string searching was invented by Bálint Dömölki in 1964 [1] and extended by R. K. Shyamasundar in 1977.

1.4. Python MultiProcess

To realize the work presented, we used the Python library **multiprocessing** that supports spawning processes using an API similar to the threading module. The multiprocessing package offers both local and remote concurrency, effectively side-stepping the Global Interpreter Lock by using subprocesses instead of threads. Due to this, the multiprocessing module allows the programmer to fully leverage multiple processors on a given machine. It runs on both POSIX and Windows. The multiprocessing module also introduces the Pool object which offers a convenient means of parallelizing the execution of a function across multiple input.

1.5. System Specification

The program execution was done within a system with the following specification:

- **CPU:** AMD Ryzen 5600 with 6 core and 12 logic cores
- **RAM:** 32 GB DDR4 3200 MHz
- **GPU:** NVIDIA RTX 3060ti with 8 GB VRAM

for the developing environment we used Pycharm 2025.2.4.

1.6. Structure Configuration

For the purpose of the problem, we used two types of experiment:

- **String generator:** we used a self defined Python function to generate a word's dataset and a string to applying the two algorithms defined before.
- **Real word dataset:** we stored inside the folder's program a text file containing real

sentences to see how the algorithm perform better and how multi processing speed up in real scenario.

2. Implementation Strategy

For memory and stability reasons, we used a number of processes minor to the number of CPU cores, equal to the latter minus two. This was done because using a number of processes equal to CPU cores results in system instability that leads to crashing and errors. That also depends to the type of operations that require intense cpu works and memory assignments.

2.1. Execution Type

In the first case, a synthetic dataset composed of randomly generated strings was created. Each string has a fixed length, and the pattern used for comparison was also randomly generated. This configuration allows testing the algorithms in a controlled environment, avoiding biases introduced by linguistic structures. In the second case, a real dataset composed of textual sentences was loaded from file. The dataset was preprocessed by removing punctuation characters and converting all text to lowercase. After preprocessing, the set of unique words contained in the dataset was extracted in order to reduce redundant computations during the comparison phase. For both execution modes, the algorithms were evaluated using two different approaches: a sequential version and a parallel version implemented through Python multiprocessing. The number of worker processes was automatically determined based on the number of available CPU cores, leaving some cores unused to avoid system overload.

2.2. Dataset chunk

In order to effectively exploit parallel execution, the input data is divided into smaller subsets, referred to as *chunks*. This operation is performed by the `chunkify` function, which splits the dataset into a predefined number of approximately equal-sized portions.

The main motivation behind this approach is to

achieve a balanced workload distribution among the available worker processes. By assigning one chunk to each process, the overall computation can be performed concurrently, reducing idle time and improving CPU utilization. Furthermore, using multiple chunks per process allows better load balancing in scenarios where the execution time associated with different elements of the dataset may vary. This strategy mitigates the impact of straggler processes and leads to more stable execution times.

2.3. String Generation

In order to evaluate the behavior of the implemented algorithms in a controlled environment, a synthetic dataset composed of randomly generated strings was created. Each string is generated by uniformly sampling characters from the lower-case English alphabet.

The generation process allows controlling both the number of strings and their length, making it possible to analyze the scalability of the algorithms independently from linguistic structures or semantic patterns that are typically present in real text.

The use of randomly generated strings ensures that the experimental evaluation is not biased by specific word distributions or recurring patterns, providing a worst-case scenario for approximate string matching algorithms such as Levenshtein distance and Bitap.

For the specific case, we used a corpus length of 50000 strings with length of this one as 120 characters, while the objective string is randomly generated with a length of 100 letters.

3. Execution and Results

For searching purpose, we confronted the multiprocess speedup within two different execution moment, related to which mode is chosed. In either case, every algorithms are reported to get the right result: the shortest distance for the Levenshtein case and the sentences which contain the word.

3.1. Random Generation

The following table shows the result of execution when the generation of string is used:

Algorithm	Multi Process	Sequential
Levenshtein	16.936 s	108.304 s
Bitap	0.695 s	3.120 s

Table 1. Random String generation execution

The resulting speed up is the following:

Algorithm	Speedup
Levenshtein	6.395
Bitap	4.491

Table 2. Speedup of random string

As we can see, in either cases the multiprocess version performs better than the classic single process, thanks to the possibilities to check different corpus of string at the same time, allowing a strong speedup and advantages in terms of program execution time.

3.2. Dataset Sampling

The following table shows the result of execution with using a real words dataset.

Algorithm	Multi Process	Sequential
Levenshtein	0.239 s	0.021 s
Bitap	0.207 s	0.022 s

Table 3. Dataset execution

and the following speedup: We can see that in

Algorithm	Speedup
Levenshtein	0.086
Bitap	0.107

Table 4. Speedup of using a dataset

this case the multi-process version of the program performs slightly worse than the sequential version. This is caused by the fact that the context switching of the CPU and the chunk change and assignment tend to increase the time needed for completion.

For future experimentation, the real advantages of using multiple processes for dealing with real sentences, using a dataset with much more sentences inside can take the lead to showing the real performance of this case.

References

- [1] B. Dömölki. A universal compiler system based on production rules. *BIT*, 8(4):262–275, Dec. 1968.
- [2] V. I. Levenshtein. Binary Codes Capable of Correcting Deletions, Insertions and Reversals. *Soviet Physics Doklady*, 10:707, Feb. 1966.